서울시립대학교
UNIVERSITY OF SEOUL

전자전기컴퓨터설계실험II

실험 후 보고서

STM32 기초 실습 및 FPGA 개별 실습

학과	전자전기컴퓨터공학부
이름 / 학번	조성현 / 2023440129
작성일	2026년 10월 13일
대상	STM32 과제1· STM32 과제2 · 4비트 곱셈기 실습 · BCD 이진수 변환 실습
검증환경	VS Code 1.137.0 / STM32 CMake/ VaporView / AMD Vivado 2026.1 XSim

소스 식별

GitHub 계정: a40093632-ai

GitHub 저장소: https://github.com/a40093632-ai/ece2-lab4

제출 태그: lab4-post-v1

커밋 해시: 2900645bfe1d799c5fc6eca0d53da63557fa74ee

1. 실험 목적

본 실험은 STM32의 GPIO 입력과 출력을 이용해 버튼으로 LED 상태 변경 간격을 조절하고, 동일한 기능을 내장 LED와 외부 LED에서 각각 확인하는 것을 목적으로 한다. FPGA에서는 4비트 곱셈기와 두 자리 BCD 변환기를 설계하여 전체 입력 조합을 테스트벤치로 검증하고, Vivado 구현 결과와 실제 보드 출력을 비교한다. 또한 STM32 디버깅 과정에서 중단점으로 실행을 멈춰 변수를 확인하고, 실행 중 값을 변경해 특정 동작 조건을 시험하는 방법을 익힌다.

2. 관련 이론

중단점과 실행 중 변수 변경

중단점(breakpoint)은 프로그램이 지정한 코드 위치에 도달했을 때 실행을 일시 정지시키는 기능이다. 디버거가 멈춘 시점에는 어떤 함수와 코드 줄을 실행 중인지 확인하고, 지역 변수와 전역 변수의 현재 값을 살펴볼 수 있다. 이후 한 줄씩 실행하면서 값이 변하는 과정을 추적하거나 실행을 재개할 수 있다. 따라서 LED가 예상과 다르게 동작할 때 입력 상태, 경과 시간, 점등 간격에 사용되는 변수를 해당 코드 위치에서 확인할 수 있다 [1].

디버거는 정지한 프로그램의 변수 값을 실행 중에 변경하는 기능도 제공한다. 변수 창에서 값을 수정한 뒤 실행을 재개하면, 프로그램은 수정된 값을 사용하여 이후 코드를 수행한다 [1]. 이번 STM32 과제의 interval_ms는 LED 상태를 변경할 시간 간격을 밀리초 단위로 저장한다. 이 값을 디버깅 중 1000에서 200으로 바꾸면, 소스 코드를 수정하거나 다시 빌드하지 않고도 LED 상태 변경 간격을 1초에서 0.2초로 바꾸어 동작을 확인할 수 있다.

3. 실험 환경 및 검증 방법

사용 장비와 연결

항목	설정
STM32 보드	NUCLEO-G474RE
STM32 입력	PC13 사용자 버튼
과제1 출력	PA5 내장 LED
과제2 출력	PB0 GPIO_OUT1_ece2에 연결한 외부 LED
STM32 개발	VS Code 빌드 및 디버깅,
STM32 확인	Serial Monitor
FPGA 장비	Combo II-DLD S75, xc7s75fgga484-1
결과 관리	Git 및 GitHub

STM32의 Serial Monitor 화면에는 ST-Link 가상 COM 포트 COM4, 9600 baud가 나타난다. 이는 디버깅 중 출력 문자열 확인용으로 사용된다.

공통 검증 순서

STM32 과제는 과제별 코드를 빌드하고 보드에 프로그램한 뒤 버튼 입력에 따른 LED 변화를 확인하였다. FPGA 실습은 각 테스트벤치로 기능을 검사하고 입력·출력 파형을 해석한 다음, Vivado에서 설계 모듈을 최상위로 설정해 합성·구현·비트스트림 생성을 수행하였다. 마지막으로 FPGA에 해당 비트스트림을 프로그램하고 스위치와 LED의 관계를 확인하였다.

실험별 설계파일

항목	설정
실습1 설계 파일	mult_4bit.v, multiply_unsigned.v, mult_4bit.xdc
실습1 검증 파일	tb.sv
실습2 설계 파일	bcd_conv.v, bcd_to_binary.v, bcd_conv.xdc
실습2 검증 파일	tb.sv

링크: 실습1 설계 및 검증 파일

링크: 실습2 설계 및 검증 파일

4. STM32 과제1: 내장 LED 간격 변경

설계 및 예상 동작

과제1에서는 PC13 버튼을 누를 때마다 PA5 내장 LED의 상태 변경 간격이 1초 → 2초 → 4초 → 8초 → 1초 순서로 순환하도록 하였다. 여기서 간격은 LED가 켜진 상태에서 꺼지거나, 꺼진 상태에서 켜지기까지의 시간이다. 버튼을 누르지 않아도 현재 선택된 간격으로 LED는 계속 동작해야 한다.

상태	저장된 간격	예상 동작
시작 시	1000ms	1초마다 LED 상태 변경
버튼 1회	2000ms	2초마다 LED 상태 변경
버튼 2회	4000ms	4초마다 LED 상태 변경
버튼 3회	8000ms	8초마다 LED 상태 변경
버튼 4회	1000ms	처음의 1초 간격 복귀

코드 동작

STM32의 주변장치 초기화가 끝나면 main.c에서 ece2_474_init()을 호출하고 printf("Hello, World! ₩n");를 실행한다. 출력 문자는 USART2를 거쳐 PC의 Serial Monitor

로 전달된다. 이 printf는 반복문에 들어가기 전에 있으므로 프로그램 시작 또는 재시작 시 한 번 실행된다.

polling_routine()은 PC13 버튼의 현재 상태 current와 이전 상태 previous를 비교한다. 이전에는 LOW였고 현재 HIGH인 순간을 새 버튼 입력으로 판단하며, 마지막으로 인정한 입력에서 200 ms 이상 지난 경우에만 간격을 변경한다. interval_ms의 초기값은 1000 ms이고, 버튼을 유효하게 누를 때마다 2000 → 4000 → 8000 → 1000 ms로 순환한다. 또한, 버튼을 누르지 않아도 현재 선택한 간격으로 LED가 계속 동작하고, 버튼을 길게 누른 동안에는 새로운 누름으로 반복 판정되지 않도록 한 구조이다.

빌드 및 보드 결과

과제1 코드를 빌드하여 오류 없이 완료되는 것을 확인한 뒤 STM32 보드에 프로그램하였다. 그림 1은 보드의 PA5 내장 LED 시연 사진이다. 버튼을 누르지 않아도 LED가 선택된 간격으로 계속 동작하였고, 버튼을 누를 때마다 상태 변경 간격이 1초 → 2초 → 4초 → 8초 → 1초 순서로 바뀌는 것을 확인하였다.

main.c의 원래 문자열은 Hello, World! 이며 프로그램을 시작하면 Serial Monitor로 한 번 전송된다. 문자열을 일시적으로 변경했을 때에는 그림 2와 같이 abc가 출력되었다.

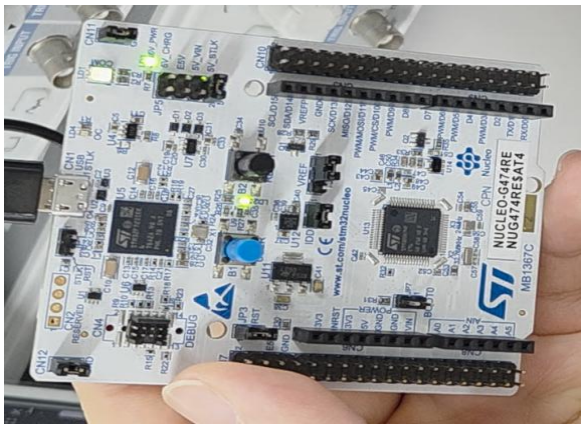


그림 1: PA5 내장 LED의 보드 시연 사진

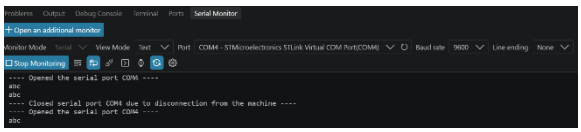


그림 2: 과제1 Serial Monitor 출력 결과

링크: PA5 내장 LED의 보드 시연 영상

5. STM32 과제2: 외부 LED 간격 변경

설계 변경

과제2에서는 과제1의 버튼 검출과 간격 순환 로직을 유지하고 LED 출력 위치만 변경하였다. 내장 LED의 PA5 대신 PB0의 GPIO_OUT1_ece2_GPIO_Port, GPIO_OUT1_ece2_Pin을 HAL_GPIO_TogglePin()에 전달하였다. PB0에는 전류 제한 저항과 외부 LED를 연결하고 LED의 반대쪽을 GND에 연결하였다.

코드 동작

과제2도 main.c의 실행 순서는 과제1과 같다. 초기화 후 printf("Hello, World! ₩n");가 USART2를 통해 PC에 한 번 출력되고, 이후 메인 루프에서 polling_routine()이 반복된다. PC13 버튼의 새 누름 판정, 200ms 재입력 제한, interval_ms의 1000 → 2000 → 4000 → 8000 → 1000 ms 순환 및 마지막 LED 변경 시각을 갱신하는 방식도 유지하였다.

달라진 부분은 LED를 반전시키는 포트와 핀이다. 과제1의 내부 핀 대신 외부의 GPIO_OUT1_ece2_GPIO_Port, GPIO_OUT1_ece2_Pin을 HAL_GPIO_TogglePin()에 전달하였다. 따라서 내장 PA5 LED 대신 PB0에 연결한 외부 LED가 선택된 간격으로 켜지고 꺼진다.

빌드 및 보드 결과

과제2 코드를 별도로 빌드하여 보드에 프로그램하고, PB0에서 전류 제한 저항을 거쳐 외부 LED와 GND로 연결한 상태를 확인하였다. 그림 3은 외부 LED를 사용한 보드 시연 사진이다. 버튼을 누르지 않아도 외부 LED가 점멸하고, 버튼을 누를 때마다 상태 변경 간격이 1초 → 2초 → 4초 → 8초 → 1초로 순환하는 것을 시연하였다.

그림 4의 Serial Monitor 화면은 과제1과 과제2가 공통으로 사용하는 main.c의 출력 문자열을 실험 중 임시로 abc로 바꾸어 확인한 기록이다.

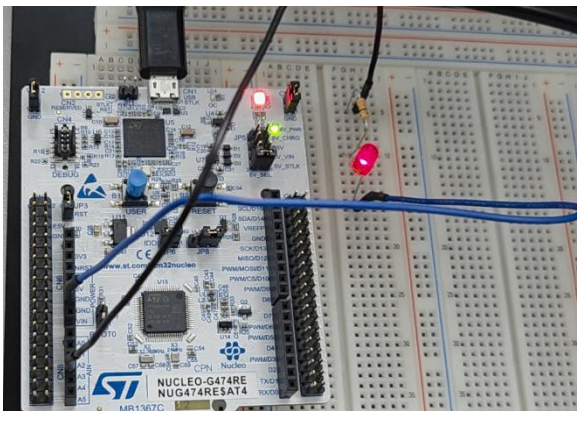


그림 3: PB0에 연결한 외부 LED의 보드 시연 사진

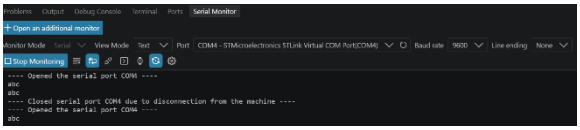


그림 4: 과제2 Serial Monitor 출력 결과

링크: PB0에 연결한 외부 LED의 보드 시연 영상

6. FPGA 실습1: 4비트 곱셈기

회로 구성

최상위 모듈 mult_4bit은 두 4비트 무부호 입력 a[3:0], b[3:0]과 8비트 출력 m[7:0]을 갖는다. 하위 모듈 multiply_unsigned에서 두 입력의 곱을 계산하고 최상위 모듈에서 입출력을 연결한다. 두 입력의 최댓값인 15와 15를 곱한 값은 225이므로 8비트 출력으로 표현할 수 있다. 이 회로는 입력이 바뀌면 출력이 결정되는 조합회로이다.

입력 a	입력 b	예상 출력 m
0	임의 값	0
3	5	15(0F)
3	12(C)	36(24)
15	15	225(E1)

Vivado 실험사진

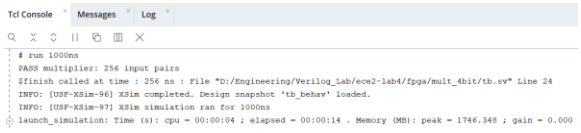


그림 5: 실습1 Vivado 시뮬레이션 PASS

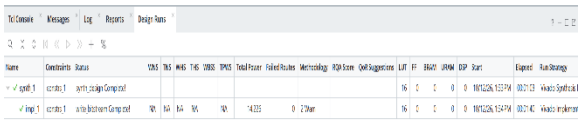


그림 6: 실습1 합성 및 구현 완료 결과

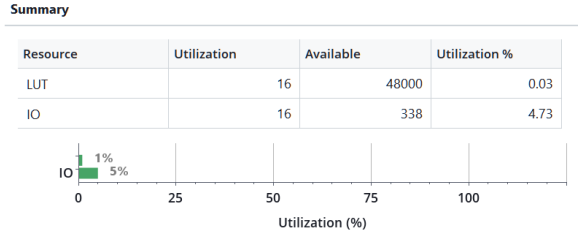


그림 7: 실습1 FPGA 자원 사용량

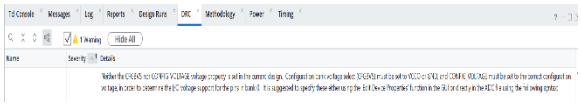


그림 8: 실습1 DRC 검사 결과

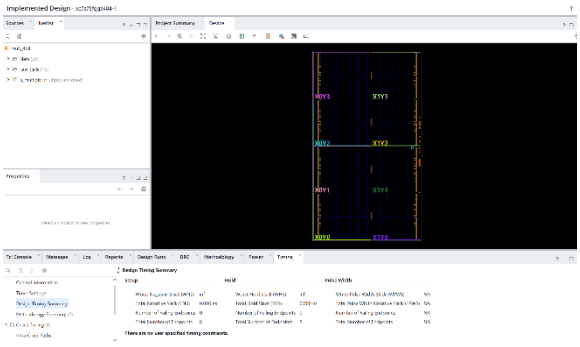


그림 9: 실습1 타이밍 분석 결과



그림 10: 실습1 Bit Hash 확인



그림 11: 실습1 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

테스트벤치 tb는 4비트 입력 a와 b에 0~15를 각각 적용하였다. 중첩 반복문을 통해 256개 입력 쌍을 검사하고, 매 조합의 곱셈기 출력 m을 예상값 a×b와 비교하였다. 값이 다르면 $fatal로 종료하도록 하였으며, 모든 검사를 마쳐 그림 5와 같이 PASS multiplier: 256 input pairs가 표시되었다.

그림 11의 Vivado 파형에서는 a=3, b=12(C)일 때 m=36(24)로 나타난다. 3 x 12 = 36인 단순 곱셈 결과와 일치한다. 또한, 그 외의 4비트 두 값에 대한 곱셈 결과도 각각 모두 일치한다.

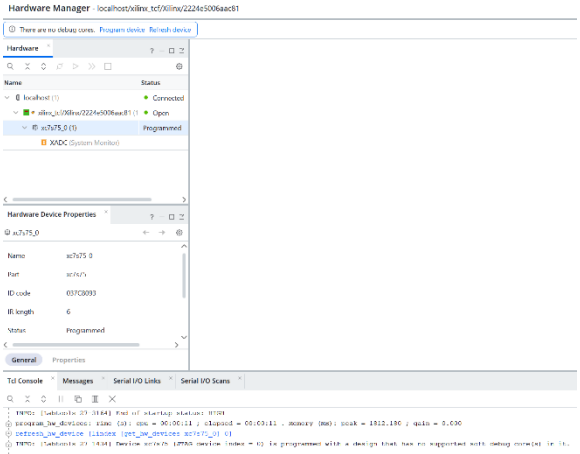


그림 12: 실습1 Program Device 확인

FPGA 보드 시연

그림 13의 보드 시연 사진은 스위치의 왼쪽 4비트(SW7~SW4)와 오른쪽 4비트(SW3~SW0)의 곱셈 결과가 LED7~LED0에서 8비트 결과 값으로 나타난 결과다. 0001과 0100의 곱셈 결과 0100 위치에 맞게 LED에 불이 들어온 것을 확인할 수 있다. 다른 4비트 간의 곱셈 결과도 예상결과와 동일하게 LED에 8비트 결과 값으로 출력되는 결과가 나타났다.

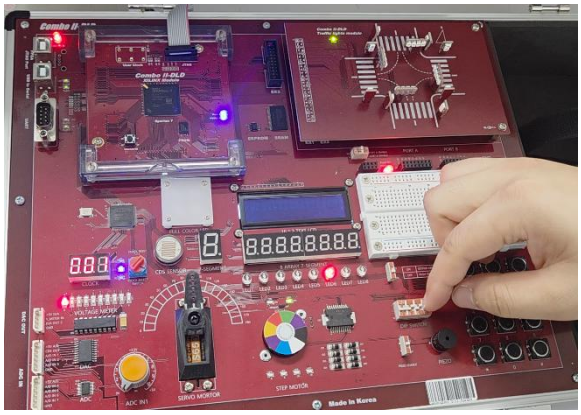


그림 13: 실습1 FPGA 보드 시연 사진

링크: FPGA 실습1 보드 시연 영상

7. FPGA 실습2: 두자리 BCD 변환기

회로 구성

최상위 bcd_conv은 두 4비트 입력 ten[3:0], one[3:0]을 하위 bcd_to_binary에 연결한다. 각 입력이 0~9인 정상 BCD이면 7비트 출력 bin[6:0]에 10 x ten + one을 표시하고 valid=1로 나타낸다. 두 자리 중 한 자리라도 A~F이면 valid=0으로 표시한다.

ten	one	예상 bin	예상 valid
0	0	0	1
3	5	35(23)	1
9	9	99(63)	1
3	10(A)	유효X	0

Vivado 실험사진



그림 14: 실습2 Vivado 시뮬레이션 PASS

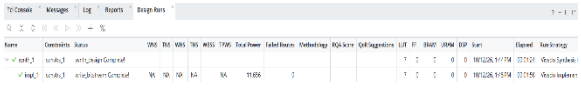


그림 15: 실습2 합성 및 구현 완료 결과

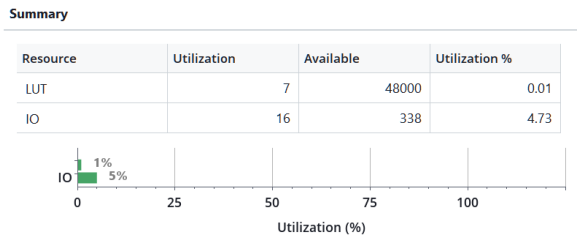


그림 16: 실습2 FPGA 자원 사용량

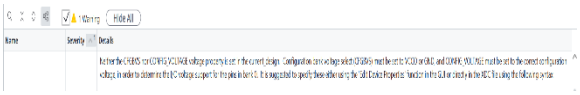


그림 17: 실습2 DRC 검사 결과

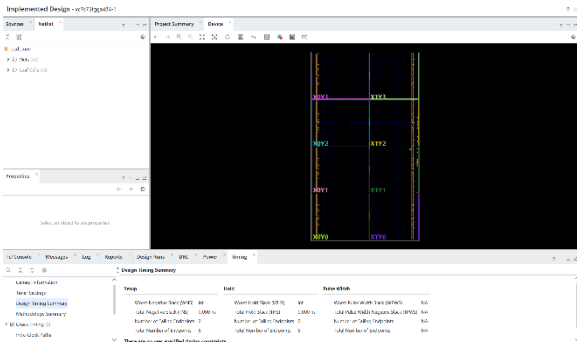


그림 18: 실습2 타이밍 분석 결과

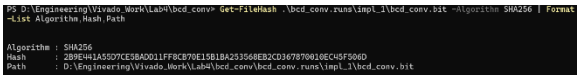


그림 19: 실습2 Bit Hash 확인

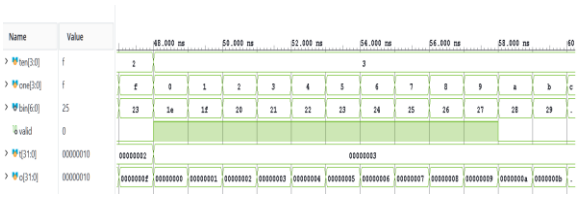


그림 20: 실습2 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

테스트벤치 tb는 십의 자리 ten과 일의 자리 one에 각각 0~15를 입력하여 총 256개 조합을 검사하였다. 두 자리 모두 0~9인 100개 조합에서는 valid=1인지와 bin=10×ten+one인지 확인하였다. 어느 한 자리라도 A~F인 나머지 156개 조합에서는 valid=0인지 확인하였다. 검사를 모두 마쳐 그림 14와 같이 PASS BCD: 100 valid and 156 invalid inputs가 표시되었다.

그림 20의 Vivado 파형에서는 ten=3, one=7일 때 bin 값에는 16진수로 23, 10진수로는 35가 정확하게 표시되었고,

유효한 값이기 때문에 valid=1로 표시되었다. 이어서 일의 자리가 A(10)인 비정상 BCD 입력에서는 valid=0으로 바뀌는 것 또한 확인할 수 있다.

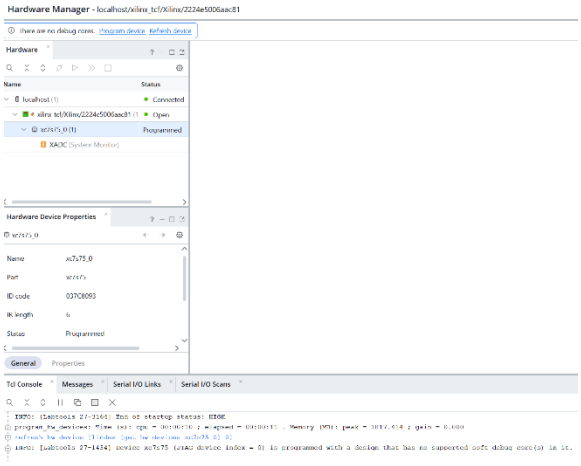


그림 21: 실습2 Program Device 확인

FPGA 보드 시연

그림 22의 보드 시연 사진은 왼쪽 스위치 4개(SW7~SW4)와 오른쪽 스위치 4개(SW3~SW0)를 두 스위치 묶음으로 하여 십의 자리와 일의 자리를 입력하고, LED6~LED0의 2진수 출력과 LED7의 valid 표시를 시연한 BCD 변환기의 동작 모습을 나타낸 것이다. 특히 0100과 0010을 입력하여 각각 십의자리와 일의자리로 하여 32+8+2=42인 2진수 값이 LED에 정확하게 나타났다. 또한 BCD에 유효한 입력 값이기 때문에 LED7은 valid=1에 맞게 불빛이 나타나는 것을 알 수 있다.

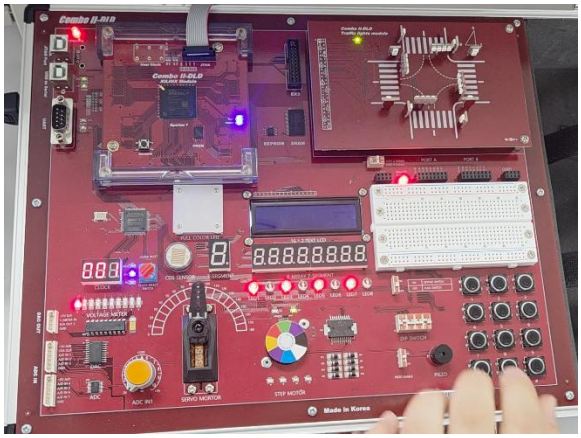


그림 22: 실습2 FPGA 보드 시연 사진

링크: FPGA 실습2 보드 시연 영상

8. 문제 발생 및 해결 과정

STM32 중단점 디버깅과 임시 변경의 구분

과제 시연 중 조교의 안내에 따라 중단점을 사용하여 STM32 프로그램의 실행을 멈추고 변수 값을 확인하였다. 버튼 입력으로만 1·2·4·8초 간격을 선택하는 정상 동작과 별도로, 중단 상태에서 interval_ms의 실행 중 값을 200 ms로 바꾼 뒤 재개하였다. LED가 더 빠르게 켜지고 꺼지는 것을 관찰하여 디버거에서 특정 동작 조건을 짧은 시간에 시험할 수 있음을 확인하였다.

이 변수 변경은 MCU가 실행 중인 상태에 적용된 것이므로 제출 코드의 정상 간격 순서를 200 ms로 수정한 것은 아니다. 이와 별개로 출력 문자열을 일시적으로 바꾸어 그림 2와 그림 4의 Serial Monitor에서 abc를 확인하였다. 문자열 변경과 실행 중 변수 변경은 방법과 남는 결과가 다르므로 구분하여 실험을 진행해야 함을 알게되었다.

9. 예상 값 · Vivado · 보드 결과 비교

실험	예상 및 판정 기준	소프트웨어 확인	실제보드 확인
과제1	PA5 LED 간격 변화	빌드 완료	내장 LED 시연
과제2	PB0 외부 LED에서 같은 간격 순환	빌드 완료	외부 LED 시연
실습1	두 4비트 입력의 8비트 곱	TB PASS, 파형 확인	스위치-LED 시연
실습2	정상 100, 비정상 156개 구분	TB PASS, 파형 확인	LED7 유효성 확인

STM32 과제의 빌드 완료는 코드가 컴파일되었다는 결과이며, LED 간격 순환은 보드 시연으로 확인하였다. FPGA의 테스트벤치는 전체 입력 조합의 논리 기능을 검사했고, 실제 보드에서는 스위치 입력에 따른 물리적 LED 출력과 핀 연결의 동작을 확인하였다.

10. Bitstream 해시 표

실습	Bitstream	SHA-256
4비트 곱셈기	Lab4_mult_4bit.bit	E68E424…
BCD 변환기	Lab4_bcd_conv.bit	2B9E441…

링크: 두 비트스트림 파일

11. 결론

STM32 과제1에서는 PC13 버튼을 사용해 PA5 내장 LED의 상태 변경 간격을 1초, 2초, 4초, 8초 순서로 조절하였다. 과제2에서는 같은 버튼·시간 제어 방식을 유지하면서 PB0의 외부 LED에서 동작을 별도로 확인하였다. 프로그램 시작 시 공통 main.c의 Hello, World! 출력과 메인 루프의 LED 제어가 서로 다른 위치에서 실행됨을 코드와 실험을 통해 확인하였다.

중단점 디버깅에서는 interval_ms의 실행 중 값을 200 ms로 임시 변경하여 정상 과제 코드를 영구 수정하지 않고 빠른 점멸 조건을 시험하였다. 코드를 디버깅하는 과정에서 중단점 사용은 코드 점검 및 시뮬레이션 관점에서 중요한 기능을 하고, 복잡한 코드를 구간별로 확인하면서 빌드를 진행하는 안전 장치의 기능도 함을 알게 되었다.

FPGA의 4비트 곱셈기는 256개 입력 쌍, BCD 변환기는 정상 100개와 비정상 156개 입력에 대한 테스트벤치 검사를 통과하였다. 각 파형의 구체적인 입력·출력 관계를 확인한 뒤 비트스트림을 생성하고 실제 보드에서 스위치와 LED의 동작을 시연하였다. 이는 앞선 3개의 lab에서와 마찬가지로 코드 작성부터 시뮬레이션, 보드 시연까지 하나의 사이클로 실험을 진행하고, 실험을 체계적이고 완성도 있게 수행하는 능력을 기르는데 도움이 되었다.

참고 문헌

[1] R. Stallman, R. Pesch, S. Shebs, et al., *Debugging with GDB: The GNU Source-Level Debugger*, 10th ed., Free Software Foundation, 2026, §§5.1, 5.2, 17.1. ISBN 978-0-9831592-3-0. https://sourceware.org/gdb/download/onlinedocs/gdb.pdf